

Spécialité : Transformation Digitale Industrielle

Module : Génie Logiciel

Compte Rendu du TP N° 01 :

Programmation Orientée Objet en Java

Filière : **TDI**

Niveau : **2^{ème}**



Effectuée par :

MOUGUERT Meryem

Encadré par :

ELBAGHAZAOUI Bahaa Eddine

Exercice 1.1 : Gestion d'animaux dans un zoo

Partie 1 : Définitions théoriques

Question 1 : Encapsulation

L'encapsulation est un principe POO qui consiste à regrouper les données (attributs) et les méthodes qui les manipulent au sein d'une même classe, tout en contrôlant l'accès depuis l'extérieur.

Dans la classe `Animal`, l'attribut `nom` est déclaré `private` : il ne peut pas être modifié ou lu directement depuis l'extérieur de la classe. L'accès se fait uniquement via la méthode publique `getNom()`, ce qui protège l'intégrité de la donnée.

Exemple dans le code :

```
private String nom;

public String getNom() {
    return nom;
}
```

Question 2 : Héritage

L'héritage permet à une classe (sous-classe) de réutiliser les attributs et méthodes d'une autre classe (super-classe), en les étendant ou en les modifiant.

Dans cet exercice :

- `Mammifere` extends `Animal` : la classe `Mammifere` hérite de tous les membres de `Animal`.
- `Oiseau` extends `Animal` : la classe `Oiseau` hérite également de `Animal`.

Le mot-clé utilisé est `extends`. Le constructeur de la super-classe est appelé avec `super(nom)`.

Question 3 : Abstraction

L'abstraction consiste à définir un comportement général (contrat) dans une classe abstraite sans fournir l'implémentation. Les sous-classes sont ensuite obligées de fournir cette implémentation.

Dans cet exercice :

- `Animal` est une classe abstraite (`abstract class Animal`).
- La méthode `faireDuBruit()` est abstraite : elle impose aux sous-classes de définir leur propre comportement sonore.

```
public abstract void faireDuBruit();
```

Question 4 : Polymorphisme

Le polymorphisme permet d'appeler la même méthode sur des objets de types différents, chacun répondant à sa façon. Ici, `tigre` et `perroquet` sont tous deux déclarés de type `Animal`, mais l'appel à `faireDuBruit()` produit un résultat différent selon l'instance réelle :

```
Animal tigre = new Mammifere("Tigre"); // grogne
Animal perroquet = new Oiseau("Perroquet"); // chante
tigre.faireDuBruit(); // -> "Tigre grogne."
perroquet.faireDuBruit(); // -> "Perroquet chante."
```

C'est le mécanisme de liaison dynamique (late binding) qui détermine à l'exécution quelle méthode appeler.

Partie 2 : Compléter le code

Question 5 : Méthode voler()

Implémentation de la méthode voler() dans la classe Oiseau :

```
public void voler() {
    System.out.println("L'oiseau vole.");
}
```

Question 6 : Appel de voler() dans main()

Pour appeler voler(), il faut utiliser une référence de type Oiseau (downcast) :

```
Oiseau perroquetOiseau = (Oiseau) perroquet;
perroquetOiseau.voler(); // -> "L'oiseau vole."
```

La référence de type Animal ne connaît pas la méthode voler(), d'où la nécessité du cast.

Partie 3 : Identification des mots-clés

extends (ex : class Mammifere extends Animal)	Héritage
@Override (annoter la méthode faireDuBruit() dans les sous-classes)	Redéfinition
abstract (devant la classe Animal et la méthode faireDuBruit())	Abstraction

Exercice 1.2 : Gestion des véhicules de transport

Partie 1 : Théorie et concepts

Question 1 : Encapsulation

Les attributs marque, modele et annee de la classe Vehicule sont déclarés private. Ils ne sont accessibles depuis l'extérieur que via les méthodes getMarque(), getModele() et

getAnnee(). De même, nombreDePortes dans Voiture et capaciteDeCharge dans Camion sont privés.

Question 2 : Héritage

La hiérarchie d'héritage est la suivante :

- Vehicule est la super-classe abstraite.
- Voiture extends Vehicule : hérite des attributs et méthodes de Vehicule, ajoute nombreDePortes.
- Camion extends Vehicule : hérite de Vehicule, ajoute capaciteDeCharge.

Chaque sous-classe appelle super(marque, modele, annee) dans son constructeur pour initialiser les attributs hérités.

Question 3 : Polymorphisme

Dans la méthode main(), maVoiture et monCamion sont déclarés de type Vehicule. L'appel à afficherDetails() est résolu dynamiquement à l'exécution selon le type réel de l'objet (Voiture ou Camion) :

maVoiture.afficherDetails(); // -> Voiture: Toyota Corolla (2021), Portes: 4

monCamion.afficherDetails(); // -> Camion: Volvo FMX (2019), Capacité: 12.5 tonnes

Question 4 : Abstraction

Vehicule est une classe abstraite car il n'est pas pertinent d'instancier un véhicule générique sans savoir si c'est une voiture ou un camion. La méthode abstraite afficherDetails() force chaque sous-classe à fournir sa propre description.

```
public abstract void afficherDetails();
```

Partie 2 : Compléter le code

Question 5 : Méthode demarrer()

Ajout d'une méthode concrète demarrer() dans Vehicule :

```
public void demarrer() {  
    System.out.println("Le véhicule démarre.");  
}
```

Les classes Voiture et Camion appellent demarrer() au début de afficherDetails() :

```
@Override
```

```
public void afficherDetails() {  
    demarrer();  
    System.out.println("Voiture: " + getMarque() + ...);  
}
```

Question 6 : Classe Moto

```
class Moto extends Vehicule {  
    private String typeDeGuidon;  
  
    public Moto(String marque, String modele, int annee, String typeDeGuidon) {  
        super(marque, modele, annee);  
        this.typeDeGuidon = typeDeGuidon;  
    }  
  
    @Override  
    public void afficherDetails() {  
        demarrer();  
        System.out.println("Moto: " + getMarque() + " " + getModele()  
            + " (" + getAnnee() + "), Guidon: " + typeDeGuidon);  
    }  
}
```

Partie 3 : Identification des mots-clés

Redéfinition : @Override

Héritage : extends

Constructeur parent : super(...)

Exercice 1.3 – Gestion de comptes bancaires

Conception du système

Le système est composé de trois classes :

- CompteBancaire : classe de base avec les attributs et méthodes communs (deposer, retirer).
- CompteCourant : hérite de CompteBancaire sans comportement supplémentaire.
- CompteEpargne : hérite de CompteBancaire et ajoute le calcul d'intérêts.
- Banque : classe principale contenant la méthode main().

Implémentation

Classe CompteBancaire

```

class CompteBancaire {
    private String numeroCompte;
    private double solde;

    public CompteBancaire(String numeroCompte, double soldeInitial) {
        this.numeroCompte = numeroCompte;
        this.solde = soldeInitial;
    }

    public void deposer(double montant) {
        if (montant > 0) { solde += montant; }
    }

    public void retirer(double montant) {
        if (montant > 0 && montant <= solde) { solde -= montant; }
        else { System.out.println("Solde insuffisant."); }
    }
}

```

Classe CompteEpargne

```

class CompteEpargne extends CompteBancaire {
    public void calculerInterets(double taux) {
        double interets = getSolde() * taux / 100;
        deposer(interets);
    }
}

```

Résultats d'exécution

Sortie console de la classe Banque :

=== Compte Courant ===

Dépôt de 500.0 € effectué. Solde : 1500.0 €

Retrait de 200.0 € effectué. Solde : 1300.0 €

Solde insuffisant. Solde actuel : 1300.0 €

=== Compte Épargne ===

Dépôt de 500.0 € effectué. Solde : 2500.0 €

Dépôt de 87.5 € effectué. Solde : 2587.5 €

Intérêts calculés au taux de 3.5% : +87.5 €

Retrait de 100.0 € effectué. Solde : 2487.5 €

Solde final : 2487.5 €

Concepts POO illustrés

Attributs numeroCompte et solde privés, accédés via getters	Encapsulation
CompteCourant et CompteEpargne étendent CompteBancaire	Héritage
Comportement commun centralisé dans CompteBancaire	Abstraction
CompteEpargne.calculerInterets() appelle deposer() héritée	Réutilisation

